

Benchmarking Rule-Based Packet Filtering on a Resource-Constrained ESP32

Spring 2026 ECE 56800 Project Report

LiteWall

John Canino (Lead), Derek Crocker, Nicholas Haas, Tariq Ihazrir, S Hassaan Mahmood

ABSTRACT

In this project, a lightweight firewall was developed on an ESP32 to explore how packet filtering can be implemented on resource-constrained embedded systems. The ESP32 was configured as a Wi-Fi Access Point, and incoming traffic from a connected client was processed using the lwIP stack. This setup provided direct access to TCP/UDP packets, allowing extraction of key fields such as source IP addresses and destination ports for filtering. A rule-based engine was implemented to classify and filter packets based on predefined conditions, such as blocking specific IP addresses or ports. The system also supports dynamic rule updates and uses rule tables stored in either SRAM or PSRAM. To evaluate system performance, network traffic was generated using Scapy with controlled packet rates and varying IP and port combinations to simulate different conditions. Performance metrics, including throughput and packet processing latency, are being analyzed to compare the trade-offs between SRAM and PSRAM. Normal traffic processing latency was between 35 and 45 μ s for SRAM and between 45 and 70 μ s for PSRAM across all rules. Worst case increased latency by approximately 17% for SRAM and 33% for PSRAM. The flood tests resulted in some packet lost albeit small (0.5-0.7%). CPU utilization increased by 792% for SRAM and 506% for PSRAM, but all tests showed promising results for the proposed application. The results of this project demonstrate that firewall functionality is feasible on embedded devices while also highlighting the performance limitations and design trade-offs associated with constrained hardware.

1 INTRODUCTION

Internet of Things (IoT) and embedded devices are showing up in more applications and industries. With this in mind, the security risks on these smaller, low-power systems are still present [1]. In terms of traditional firewalls, they are designed with high-performance hardware in mind. Therefore, there is always the question of how effective are firewalls on microcontrollers. That makes studying packet filtering on these platforms a necessary step before any design decisions are defined [5]. If packet filtering is skipped, embedded and IoT devices become easy targets, leading to slow responses, unexpected crashes, and data ending up in the wrong hands.

2 BACKGROUND

Most firewall implementations are found on routers or servers with significant computational resources, allowing them to handle complex filtering rules and high network traffic [3]. However, embedded systems like the ESP32 have limited memory and processing power, which makes traditional firewall approaches difficult to implement.

At the same time, the embedded systems have enabled many modern applications, leading to an increase in low-power connected devices. Additionally, this growth has introduced a range of security vulnerabilities, making embedded systems attractive targets for malicious attacks. As a result, strong cybersecurity mechanisms have become necessary [4]. For instance, Bertoli et al. have developed a lightweight packet filtering approach that uses machine learning techniques to classify network traffic on resource-constrained embedded devices. The results show that their machine learning approach makes the system run by blocking unwanted and potentially harmful traffic before it gets processed [2]. However, their machine learning models classify packets, requiring labeled training data and introducing inference overhead on every packet that passes through the device.

3 PROBLEM STATEMENT

The main challenge of this project is to build a functional and efficient packet-filtering firewall on an ESP32 with limited resources. The team's goal is to maintain acceptable performance despite these constraints. This project specifically looks at how packet processing speed, data handling capacity, and overall system performance are impacted by accessing filtering rules located on different memory types (SRAM and PSRAM).

4 PROPOSED APPROACH

The proposed approach consist of a lightweight firewall built on an ESP32 that runs in Wi-Fi Access Point (AP) mode as shown in the high-level architecture on Figure 1. In terms of network architecture, a client device connects and simulates network traffic to the ESP32 network stack. The lightweight IP (lwIP), an open source TCP/IP stack library, handles the network stack, giving direct access to incoming TCP/UDP packets at the transport layer. Hence, making header inspection straightforward without any need for decryption. As far as firewall logic, the code implemented on the ESP32 was designed to leverage the lwIP library to fire a callback every time a packet arrives. Additionally, lwIP parses packet header information, including source IP and destination port. This information is then consumed by a rule engine, where the packet is checked against a set of defined rules and it is either allowed or blocked. Finally, the code is designed to log each decision and track metrics such as total packets handled and how long processing takes.

4.1 Packet Processing using lwIP

The design processes packets by bypassing the Wi-Fi Protected Access (WPA) encryption by handling packets directly through the lwIP stack. This allows direct access to TCP/UDP packets, eliminating the need for further decryption and setup. In this case, the

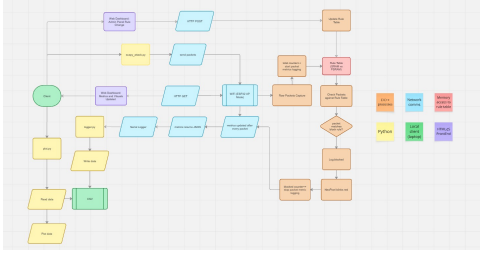


Figure 1: High-level Architecture

focus is on extracting the source IP address and destination port number since these are needed for applying the filtering rules. Once this information is extracted, it is passed to the rule engine to decide whether the packet should be allowed or blocked. At the same time, metrics are logged before moving on to the next packet. This approach is better suited for an embedded system like the ESP32, offering a more reliable packet inspection method than promiscuous mode.

4.2 Rule Engine

Memory Table

Active Table: SRAM

Switch Table

Current Rules

ID	IP	Port	Protocol	Action
0	1.161.141.236	0	TCP	BLOCK
1	103.109.187.71	0	TCP	BLOCK
2	0.0.0.0	90	TCP	BLOCK
3	0.0.0.0	120	UDP	BLOCK
4	155.138.0.0	80	TCP	BLOCK
5	0.0.0.0	80	UDP	BLOCK
6	161.29.72.10	90	TCP	BLOCK
7	168.78.62.53	0	TCP	BLOCK

Figure 2: Example Rule Table

The rule engine used in this paper classifies incoming packets based on a set of predefined filtering rules. When a packet is received through the lwIP raw callback, fields like source IP address, destination port, and protocol are extracted and passed to the rule engine. In this scenario, the engine iterates through the rule table and checks if any active rule matches the packet. Rules can match by IP only, port only, or both IP and port combined. Therefore, if

a match is found, the packet is blocked; otherwise, it is allowed through. In terms of support, the rule table allows for initialization, adding new rules, removing rules by index, and clearing all rules through a web dashboard user interface. This allows firewall behavior to be updated while the system is running, either through the web dashboard or a comma separated value (CSV) file upload. For benchmarking, the rule table can be stored in either internal SRAM or external PSRAM. The difference between both storage methods, is that SRAM is allocated statically, whereas PSRAM is allocated dynamically. This setup allows comparison of memory access latency and rule lookup performance between the two storage options during packet processing.

4.3 Memory Design

In relation to performance trade-offs on the ESP32, the rule table was implemented using both internal SRAM and external PSRAM. For example, SRAM offers faster access and lower latency but limited capacity, while PSRAM provides more space at the cost of slower access. The rule engine supports both memory types and can switch between them at runtime using a flag. At boot, the SRAM table is allocated as a static array inside the rule engine class. As previously mentioned, PSRAM is allocated at boot using heap caps malloc with the SPIRAM flag, which directs allocation to the external SPI-connected RAM. A toggle in the web dashboard switches between the two tables without restarting the system. During packet processing, the rule engine performs lookups directly from whichever table is active. This allows comparison of how memory location impacts rule lookup latency and cycles. By varying the number of rules stored in each table, the system can also be tested for scalability, providing insight into the trade-offs between access speed and storage capacity in an embedded firewall.

4.4 Traffic Generator (Scapy)

A python package, Scapy, was used on a client device to simulate network traffic while connected to the ESP32 access point. In terms of packets, they were built from a CSV file containing source addresses, destination ports, and protocols, generating TCP and UDP packets with random source ports and a fixed ESP32 destination IP. This setup made it possible to test IP-based, port-based, and combined firewall rules without needing multiple physical devices. Two traffic modes were used normal load and flood. Normal load sent one packet per entry per cycle, while flood sent 75 copies of each packet with a fixed delay between them to simulate high traffic. Finally, the packet list was shuffled between each cycle to vary the arrival order, providing a repeatable way to test the firewall in a controlled setup and collect data.

4.5 Performance Monitoring

Performance monitoring was added to evaluate the ESP32 firewall's handling of incoming traffic. The dashboard tracked key metrics: total packets, TCP packets, UDP packets, blocked threats, throughput, processing latency, CPU usage, and memory usage. These values provided insight into whether the rule engine functioned correctly and how the system behaved under different traffic conditions. The dashboard updated in real time, displaying packet counts and changes in traffic over time. Throughput was measured in packets

per second, and latency was measured in microseconds to estimate packet processing and classification time. These measurements could be used to compare configurations, such as storing rules in SRAM versus PSRAM.

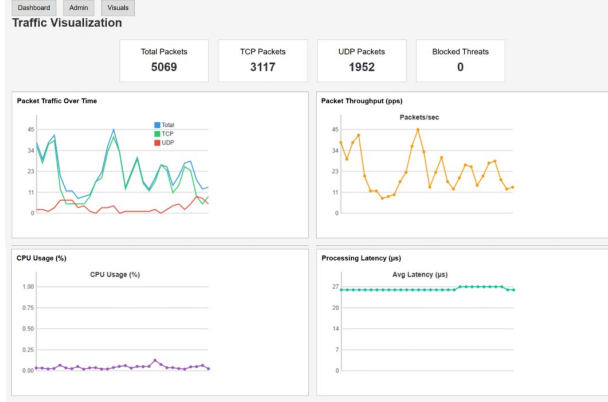


Figure 3: Dashboard Visuals

4.6 Experimental Setup

The experimental setup consists of an ESP32 Feather V2 configured as a WiFi access point and a laptop connected to it as the client. Traffic was generated using Scapy scripts that simulate up to 100 source IPs sending TCP packets to the ESP32. Three traffic patterns were evaluated: normal traffic at 100 pps with 10% of sources blocked, worst-case traffic where every packet matches the last rule in the table forcing a full traversal, and flood traffic at maximum rate to stress the packet pipeline. Rule tables were loaded via CSV upload before each run. Each benchmark ran for 180 seconds with metrics logged over serial at 1 second intervals. Rule lookup latency, processing latency, packets per second, blocked threat count, and packet loss rate were recorded across ten test configurations covering block rule counts of 10, 50, and 100 (out of a total 100 rules) under both SRAM and PSRAM storage. This setup provides a controlled and repeatable environment for comparing memory type and rule table size effects on firewall performance.

Table 1: Benchmark Configuration

Set	Match Behavior	Duration (s)	Traffic Pattern	Approx. Packet Rate
1-2	10% block, 90% pass	180	Normal	100
3-4	50% block, 50% pass	180	Normal	100
5-6	100% block	180	Normal	100
7-8	100% block (last rule match)	180	Worst-Case	100
9-10	50% block, 50% pass	180	Flood	1000

5 EVALUATION

With the given experimental setup, SRAM and PSRAM tests were run with various block rule counts and traffic patterns. During operation of the LiteWall AP, metrics were captured and stored in a JSON string for use with the asynchronous web server’s API

endpoints. During our testing, these metrics were exported to the serial bus with a constant period, and these metrics were then read and analyzed with external Python scripts to produce the plots described here. This choice was made to limit additional storage, memory, and CPU utilization while enabling simple deactivation of the prints when testing was completed.

5.1 SRAM vs PSRAM with Normal Traffic Patterns

The first test was run with normal traffic (approximately 100 packets per second) using test cases with 10%, 50%, and 100% block rules with rules saved in either SRAM or PSRAM. The throughput characterization was made through latency measurements as seen in Figure 4.

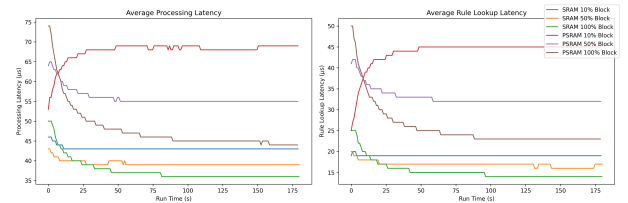


Figure 4: Latency Results with Normal Traffic

The SRAM stored rule table outperformed the PSRAM rule table in all tests, with even the most strict block rule of the SRAM test outperforming the least strict block rule of the PSRAM test. This is a testament to the processing cost of accessing the external memory during processing, making it suitable only for large block lists that are less frequently accessed or less performance dependent packet processing.

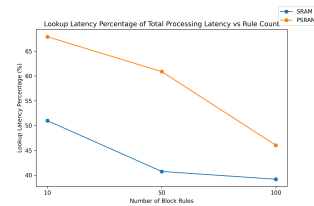


Figure 5: Lookup Latency Percentage with Normal Traffic

The percent of time spent specifically on lookup latency shown as a fraction of total processing latency is displayed in Figure 5. The PSRAM rule table is again seen to be more costly to packet throughput as the cost of lookup is higher, and SRAM sees a much lower cost difference between block percentages below 100%.

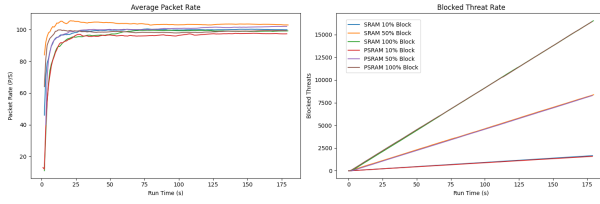


Figure 6: Packet Data Results with Normal Traffic

The packet rates for this set of tests are shown in Figure 6 alongside the cumulative blocked threats. The packet rate (moving average) is highest in the PSRAM 100% block and lowest in the SRAM 50% block. This is somewhat unexpected, and it is worth noting that the scapy attack script sent 100 packets per second to the Lite-Wall AP. The blocked threat rate plot shows the blocks occurred according to the preset block rules.

5.2 SRAM vs PSRAM with Worst-Case Traffic Patterns

Worst-case traffic patterns resulted in full traversals of the rule tables in both SRAM and PSRAM. For these tests, as in the previous baselines, the rule table was kept the same (except for location), and the packets sent were all blocked by the last rule.

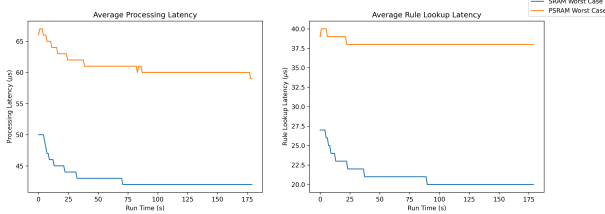


Figure 7: Latency Results with Worst-Case Traffic

Latency results from the worst case tests are shown in Figure 7 where processing latency increased from 36 μ s (SRAM normal traffic with 100% block rule) to 42 μ s (an increase of 17%). The latency of the PSRAM test increased from 45 μ s with normal traffic and 100% block to 60 μ s under this worst-case condition (an increase of 33%). The initial spike seen is likely due to the setting up of the TCP connection.

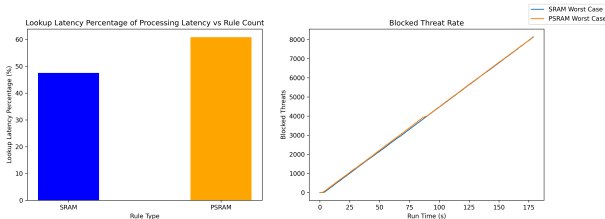


Figure 8: Packet Data Results with Worst-Case Traffic

The lookup latency, as seen in Figure 8, shows the lookup latency of this particular test accounts for almost 50% of all processing

latency in the SRAM case and for over 60% in the PSRAM case. Compared to the latency breakdown of Figure 5 for the normal traffic conditions, this is still less than the lookup percentages of the 10% block condition. With only 10% of packets blocked, 90% have to traverse the entire table, putting these two on the same magnitude of throughput reduction for rule lookup.

5.3 SRAM vs PSRAM with Flood Traffic Patterns

In the flood tests, the packet rate was increased to near 1000 packets/second as seen in Figure 9. Rather than latency which is measured by the board on a packet-by-packet basis, other factors such as CPU utilization and packet loss were observed to have been affected by the increased rate of packets.

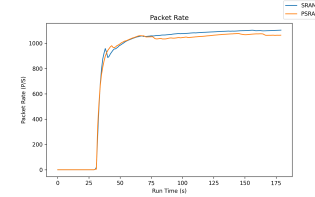


Figure 9: Packet Rate During Flood Test

The Figure above shows the idle period of 30s followed by a plateau near the 1000 pps figure that was targeted.

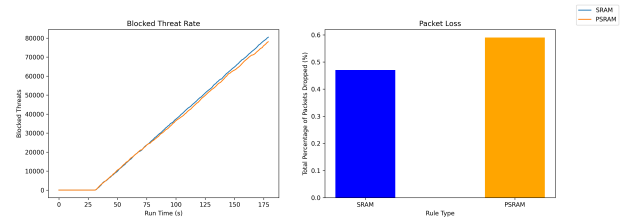


Figure 10: Packet Data Results with Flood Test

Figure 10 shows the blocked threat rate remained constant throughout, and the random deviations from the mean reflect the randomness inserted into the packet injection. The figure also shows the loss of packets under these stress tests, with PSRAM rule storage resulting in higher overall packet drop rates. While these were small rates of lost packets, they account for metrics variations along with the CPU utilization of Figure 11

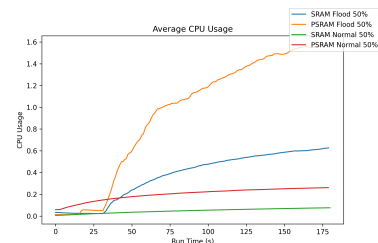


Figure 11: CPU Utilization Comparison

CPU loading is another point of comparison between the normal and flood traffic pattern tests. During the flood test, as shown in Figure 11 above, CPU utilization continues to increase as the system receives more packets than it can handle in a near simultaneous flood of traffic. Normal operation approaches a lower limit with CPU usage with maximum values as follows: 0.07% for SRAM, 0.26% for PSRAM, 0.625% for SRAM flood, and 1.578% for PSRAM flood. All tests did remain below 2% CPU utilization throughout the test period although there were utilization increases of up to 800%.

6 DISCUSSION

The strongest outcome of this project is demonstrating that a fully functional and modular firewall can be implemented on a limited platform such as the ESP32. Using the lwIP stack proved to be an effective design decision as it simplified packet parsing while providing stable access to TCP and UDP headers. This allowed the overall filtering pipeline to be stable and trackable. Basing the rule engine on linear traversal with port matching made it easy to validate Scapy-generated traffic. Moreover, the dynamic rule alteration capabilities demonstrated the flexible behavior of the firewall. Another overall success was the experiments using the Scapy library. Using this application, traffic was varied in terms of packet count, packet rates, ports, and IP addresses, making it possible to create several important test scenarios and identify possible performance bottlenecks. The dashboard also proved to be effective in visualizing throughput, latency, and system utilization. The dashboard was instrumental in gauging how certain design decisions affected system performance. Lastly, the decision to utilize SRAM in this system's architecture provided lower latency and better throughput over PSRAM.

However, certain inefficiencies and unexpected results were identified during experimentation. In the linear rule lookup, certain scenarios caused a substantial increase in latency. This occurred when the matching rules were at the end of the table or when packets needed to undergo a full traversal. This highlights a scalability issue where as the rule table gains more entries, overall performance will degrade. Under worst-case conditions, both memory configurations handled the added lookup latency well, with SRAM settling near 42 μ s processing latency and PSRAM near 60 μ s, a 17% and 33% increase, respectively. The higher effects of this delay on PSRAM were consistent with the higher access overhead of external SPI-connected memory. While the overall flood results demonstrate that the system is largely capable under high load, PSRAM showed greater overhead with increased packet loss and slightly lower sustained packet rates.

There are important limitations in the system's current state that reduce applicability in other environments. The main restraint is that this firewall is only capable of processing traffic directed towards the ESP32, making it unable to serve as a network-wide firewall. The rule engine does not scale well for larger rule sets. More advanced data structures could be explored to improve efficiency, but would likely draw too much processing power from this system's limited resources. To allow for repeatability in this study's experimentation, Scapy was utilized to generate traffic. However, the controlled data streams do not reflect realistic network traffic,

which can feature malformed packets, mixed protocols, or the retransmission of failed packet sends. Due to hardware limitations, deeper protocol inspection was not reliable. As such, this system is incapable of detecting advanced threats that would require payload inspection or other methods. Finally, measurement accuracy under flood testing suffers as packet drops and buffering distort latency metrics.

The objective of this project was to explore whether an effective firewall could be implemented on a resource-restricted platform, and the results demonstrate that it is feasible. By leveraging the ESP32 board in Wi-Fi access point mode along with the lwIP stack, a firewall was successfully developed that filters simulated traffic based on a rule table while providing system performance metrics to a dashboard. Through analysis of memory design, it was established that SRAM consistently delivered lower latency and higher throughput while PSRAM provided greater storage capacity at lower performance. The experimental results also revealed how the system handles different traffic conditions. Under normal traffic, the firewall performed with low latency and stable throughput. Under flood conditions, packet loss increased alongside CPU utilization, with PSRAM showing greater sensitivity to high packet rates than SRAM. Despite the increase, the ESP32 handled the flood test relatively well, keeping the peak CPU utilization below 2% and dropping only a handful of packets. This project serves as a study of how to introduce firewall concepts into embedded systems, demonstrating that basic firewall features are achievable and that more advanced features are possible with more sophisticated hardware.

Improvements in the current design can be made to enhance performance, scalability, and functionality. For instance, the current linear rule lookup mechanism does not scale well in terms of lookup times as the number of rules increases. Future iterations could utilize more advanced data structures such as hash tables or tree-based searches to address this. Another improvement is better memory use by using a hybrid approach, where high-priority rules are stored in SRAM and low-priority rules are offloaded to PSRAM, balancing speed and capacity. A limitation worth addressing is the system's inability to inspect traffic not directed at the ESP32. Therefore, adopting promiscuous mode or configuring the ESP32 to act as a gateway device could address this limitation, though either approach would require more capable hardware to be practical. Although flood testing demonstrated that the system handles high packet rates with low packet loss, further stability improvements could be achieved through rate limiting and scheduling, particularly to reduce the burst sensitivity observed in PSRAM configurations. The last improvement area is measurement and evaluation. This can be accomplished by generating more realistic traffic patterns to more thoroughly stress test the system.

7 POTENTIAL IMPACTS

This project demonstrates the practical potential of a firewall system on hardware with limited computing resources. As IoT devices continue to be deployed across a wide variety of fields, more devices remain vulnerable to potential attacks. The results of this study provide a solution, where a low-power micro controller such as the ESP32 can maintain essential firewall operations. A possible impact of this work is its application in low-cost edge security.

Often, embedded systems operate in environments where full-scale firewall hardware is prohibited due to power or size limitations. The low-power and small form factor firewall of this project could act as the first defensive measure for smart home systems or small IoT networks. This project also represents a possible alternative to established edge-based cyber security systems, where a centralized cloud is involved. Instead, this embedded firewall could allow for a distributed security model across several endpoint devices. As computing hardware continues to improve, security systems based on this paper’s design will be able to feature more advanced techniques such as machine learning assisted filtering.

8 CHANGE OF PLAN

One major change in plan was moving away from the original idea of using ESP32’s promiscuous mode. Research showed several issues that would take a lot of configuration to work around. For instance, promiscuous mode captures all packets on the network, but with WPA enabled, only 802.11 frame headers and MAC addresses are visible, while IP and TCP headers stay encrypted. Therefore, three options were considered. First, running the ESP32 in STA + AP mode with IP forwarding and NAT enabled. Second, using an open (unencrypted) access point with promiscuous mode for benchmarking. Lastly, using the lwIP stack to access incoming TCP and IP headers directly. In the end, lwIP was chosen since it gives direct access to the IP/TCP processing path where the rule engine sits, without the extra complexity of routing. Using the ESP32 as a router would have required IP forwarding, NAT, and encrypted traffic handling, which was too much complexity for the project’s scope. Finally, deciding on lwIP includes the benefit that only packets addressed to the ESP32 are processed, so forwarding is not possible and the system operates strictly as an on-device firewall.

9 EMBEDDED SYSTEMS ASPECTS

In this current design, the ESP32 is deployed as an application firewall enforcing rule-based packet filtering at the Wi-Fi access point layer. In terms of hardware specifications, ESP32 is a resource-constrained embedded platform with a dual-core LX6 processor running at 240MHz, 520KB of on-chip SRAM, and 2MB of external PSRAM. This makes it a realistic setup for looking at tradeoffs in embedded system design. Packet interception is done using the lwIP raw callback API, which the network stack calls every time a packet is received, matching the callback-based design covered in class and labs. The ESP32 also runs an asynchronous web server that provides a dashboard and REST API over HTTP, so it has to handle packet filtering, web serving, and metric tracking all at the same time. This shows the challenge of managing limited resources on embedded hardware. A key part of the design is where the firewall rule table is stored, either in SRAM or PSRAM depending on the test setup, and benchmarking is used to measure the difference in latency. This ties into the memory tradeoffs discussed in class, where faster on-chip memory is limited in size. Memory is limited since the ESP32 only has 520KB of SRAM, which is shared between the Wi-Fi stack, web server, packet handling, and rule storage.

10 TASK BREAKDOWN

Table 2: Task Breakdown

Area	Task	Lead
1 Research	Read papers and install tools	Everyone
	Set up GitHub repo	Derek, Hassaan
	ESP32 booting in AP mode	John, Tariq
2 Prototype	Packet capture printing to serial	Derek, John
	Parse IP and port from packet	John, Derek, Hassaan
	One hardcoded block rule working	Tariq, John
	Scapy sends first test packet	Hassaan, Nick
3 Integration	Expand rule table to multiple rules	John
	SRAM and PSRAM rule storage	John, Tariq
	Latency and heap metrics	Derek
	JSON metrics endpoint	Nick, Derek
	Web dashboard live	Nick, Derek, Hassaan
	End to end system test	Everyone
4 Benchmarking	Baseline benchmark	John, Nick, Derek
	SRAM vs PSRAM benchmark	John, Nick, Derek
	Flood stress test	John, Nick
	Generate all graphs	Nick
	Start documentation	Everyone
5 Report	Write all report sections	Everyone
	Build presentation slides	Everyone
	Rehearse live demo	Everyone
6 Finals	Finalize Report	Everyone
	Submit report	John
	Deliver presentation and live demo	Everyone

11 RELATION TO OTHER PROJECT

Not applicable.

REFERENCES

- [1] Luigi Atzori, Antonio Iera, and Giacomo Morabito. 2010. The Internet of Things: A Survey. *Computer Networks* 54, 15 (2010), 2787–2805.
- [2] Gustavo de Carvalho Bertoli, Gabriel Victor C. Fernandes, Pedro H. Borges Monici, César H. de Araujo Guibo, Aldri Luiz dos Santos, and Lourenço Alvez Pereira Júnior. 2024. Design and implementation of intelligent packet filtering in IoT microcontroller-based devices. *Journal of Internet Services and Applications* 15, 1 (Aug. 2024), 289–301. 10.5753/jisa.2024.3835
- [3] D. Brent Chapman and Elizabeth D. Zwicky. 1995. *Building Internet Firewalls*. O’Reilly Media.
- [4] A. Grau. 2026. The Internet of Things Needs Firewalls Too. <https://www.electronicdesign.com/technologies/communications/article/21796208/the-internet-of-things-needs-firewalls-too>. Accessed: 2026-03-20.
- [5] Rodrigo Roman, Jianying Zhou, and Javier Lopez. 2013. On the Features and Challenges of Security and Privacy in Distributed Internet of Things. *Computer Networks* 57, 10 (2013), 2266–2279.